

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : STI

Nama Kelompok : Sistem Terintegrasi

1. 13524008 / Agatha Tatianingseto
2. 13524030 / Irvin Tandiarrang Sumual
3. 13524074 / Bernhard Aprillio Pramana
4. 13524096 / Moreno Syawali Ganda Sugita
5. 13524110 / Jennifer Khang



STI - Sistem Terintegrasi

1. Diagram Kelas

(Dapat diDiagram Class.puml pada folder root Tugas Besar)

Gambar 1.1 Diagram Kelas

2. Deskripsi Kelas

2.1. Tile

Tile merupakan kelas abstrak yang menjadi dasar seluruh jenis petak pada papan permainan. Kelas ini menyimpan identitas umum setiap petak, yaitu `tileID_`, `letterCode_`, `tileName_`, dan `colourBlock_`. Method getter seperti `getTileID()`, `getLetterCode()`, `getTileName()`, dan `getColourBlock()` digunakan agar kelas lain seperti `Board`, `GameController`, atau `View` dapat mengakses informasi petak tanpa mengubah state internalnya. Method `getDetail()` menyediakan tampilan informasi dasar petak, sedangkan `onLand()` dideklarasikan sebagai pure virtual function agar setiap turunan petak wajib mendefinisikan perilakunya masing-masing ketika pemain mendarat di petak tersebut. Dengan desain ini, Tile menjadi pusat penerapan polymorphism karena controller cukup memanggil `onLand()` dari objek bertipe `Tile*`, lalu perilaku aktual akan mengikuti kelas turunannya.

2.2. PropertyTile

PropertyTile merupakan kelas turunan dari Tile yang merepresentasikan petak yang dapat dimiliki pemain. Kelas ini menyimpan data kepemilikan dan status properti melalui atribut `ownerUsername_`, `propertyStatus_`, `purchasePrice_`, `mortgageValue_`, dan `level_`. Selain itu, kelas ini juga menyimpan efek festival melalui `festivalMultiplier_` dan `festivalDuration_`. Method seperti `setOwnerUsername()`, `setPropertyStatus()`, `isOwnedByBank()`, `isMortgaged()`, dan `resetAfterBankruptcyToBank()` digunakan untuk mendukung alur pembelian, gadai, tebus, dan kebangkrutan. Kelas ini menjadi parent bagi `StreetTile`, `RailRoadTile`, dan `UtilityTile`, sehingga seluruh properti memiliki mekanisme dasar yang sama, tetapi tetap dapat memiliki cara perhitungan sewa dan aksi pendaratan yang berbeda.

2.3. ActionTile

ActionTile merupakan turunan dari Tile yang menjadi abstraksi umum untuk petak non-properti. Kelas ini digunakan sebagai parent bagi petak yang tidak dapat dimiliki pemain, seperti kartu, pajak, festival, dan petak spesial. Karena seluruh petak aksi memiliki warna default dan

tidak memiliki atribut kepemilikan, ActionTile cukup meneruskan data dasar ke Tile. Relasi ini memisahkan petak yang memiliki mekanisme ekonomi kepemilikan dari petak yang hanya menjalankan efek ketika pemain mendarat.

2.4. PAMTile

PAMTile merupakan kelas konkret dari UtilityTile yang merepresentasikan petak PAM. Sama seperti PLNTile, kelas ini menggunakan kembali seluruh mekanisme dari UtilityTile, termasuk kepemilikan otomatis, status gadai, dan perhitungan sewa berdasarkan hasil dadu. Pemisahan PAMTile dari PLNTile membantu menjaga representasi objek papan tetap jelas dan memudahkan apabila di masa depan tiap utility membutuhkan efek khusus.

2.5. PLNTile

PLNTile merupakan kelas konkret dari UtilityTile yang merepresentasikan petak PLN. Kelas ini tidak menambahkan atribut atau mekanisme baru, tetapi tetap dibuat sebagai kelas terpisah agar setiap petak pada papan dapat direpresentasikan secara eksplisit sesuai jenisnya. Method onLand() pada kelas ini hanya meneruskan proses ke UtilityTile::onLand(), sehingga seluruh mekanisme kepemilikan otomatis dan pembayaran sewa utility tetap mengikuti perilaku umum utility.

2.6. PropertyTile

Jelaskan secara umum tanggung jawab dari kelas ini dan bagaimana relasinya dengan kelas kelas lain. Tanggung jawab kelas dapat dijelaskan dengan menyinggung peran dari masing masing method dan atribut serta bagaimana hubungan yang terjadi antar kelas dapat membantu proses tanggung jawab tersebut.

2.7. RailroadTile

RailRoadTile merupakan turunan dari PropertyTile yang merepresentasikan petak stasiun. Berbeda dari StreetTile, railroad tidak dibeli melalui prompt, tetapi langsung menjadi milik pemain pertama yang mendarat di petak tersebut. Atribut static railRoadRentPrices_ menyimpan tabel harga sewa berdasarkan jumlah railroad yang dimiliki pemain. Method calculateRentPrice() menghitung biaya sewa dari jumlah railroad pemilik. Pada onLand(), jika railroad masih berstatus BANK, kepemilikan langsung diberikan kepada pemain. Jika railroad tergadai, tidak ada sewa. Jika dimiliki pemain lain, method mengembalikan hasil yang memicu alur pembayaran sewa oleh controller.

2.8. StreetTile

StreetTile merupakan turunan dari PropertyTile yang merepresentasikan lahan/kota pada permainan. Kelas ini memiliki atribut khusus rentPricePerLevel_ untuk menyimpan harga sewa berdasarkan level bangunan dan buildPrice_ untuk menyimpan biaya pembangunan rumah atau hotel. Method calculateRentPrice() menghitung sewa berdasarkan level bangunan, status monopoli color group, dan efek festival. Method upgradeBuilding() digunakan untuk menaikkan level bangunan. Pada onLand(), kelas ini menangani beberapa kondisi utama: jika petak milik sendiri maka tidak terjadi pembayaran, jika milik pemain lain maka memicu pembayaran sewa, jika belum dimiliki maka pemain dapat membeli atau memicu lelang. Dengan demikian, StreetTile berelasi langsung dengan Player, CommandInterface, dan GameViewInterface dalam proses pembelian dan pendaratan.

2.9. UtilityTile

UtilityTile merupakan turunan dari PropertyTile yang menjadi parent bagi petak utilitas seperti PLN dan PAM. Kelas ini menyimpan utilityFactor_, yaitu faktor pengali sewa berdasarkan jumlah utility yang dimiliki pemilik. Method calculateRentPrice() menghitung sewa dengan mengalikan hasil dadu pemain dengan faktor utility dan multiplier festival. Pada onLand(), utility yang belum dimiliki otomatis diberikan kepada pemain yang pertama mendarat, sedangkan utility milik pemain lain akan memicu pembayaran sewa. Relasinya dengan Player digunakan untuk mengubah kepemilikan dan menambahkan properti ke inventory pemain.

2.10. FestivalTile

FestivalTile merupakan turunan dari ActionTile yang merepresentasikan petak Festival. Ketika pemain mendarat di petak ini, method onLand() memeriksa apakah pemain memiliki properti. Jika tidak memiliki properti, proses selesai tanpa efek. Jika pemain memiliki properti, method mengembalikan OnLandResult::Festival agar controller menjalankan alur pemilihan properti dan pengaktifan multiplier festival. Relasi dengan PropertyTile terjadi secara tidak langsung karena efek festival disimpan pada properti melalui festivalMultiplier_ dan festivalDuration_

2.11. TaxTile

TaxTile merupakan turunan dari ActionTile yang menjadi parent bagi semua petak pajak. Kelas ini tidak menyimpan mekanisme spesifik karena setiap jenis pajak memiliki aturan berbeda. Dengan adanya TaxTile, IncomeTaxTile dan LuxuryGoodsTaxTile tetap berada pada hierarki yang sama sebagai petak aksi yang berhubungan dengan pembayaran ke bank.

2.12. SpecialTile

Jelaskan secara umum tanggung jawab dari kelas ini dan bagaimana relasinya dengan kelas kelas lain. Tanggung jawab kelas dapat dijelaskan dengan menyinggung peran dari masing masing method dan atribut serta bagaimana hubungan yang terjadi antar kelas dapat membantu proses tanggung jawab tersebut.

2.13. CardTile

CardTile merupakan turunan dari ActionTile yang menjadi parent untuk petak kartu, yaitu ChanceTile dan CommunityChestTile. Kelas ini berperan sebagai pengelompokan logis agar semua petak kartu berada dalam satu hierarki. Dengan adanya CardTile, controller dapat mengenali bahwa petak turunannya akan berhubungan dengan pengambilan kartu, walaupun jenis deck yang digunakan berbeda.

2.14. IncomeTaxTile

IncomeTaxTile merupakan turunan dari TaxTile yang merepresentasikan Pajak Penghasilan atau PPH. Kelas ini memiliki atribut static flatCost_ dan taxPercentage_ yang nilainya berasal dari konfigurasi tax.txt. Method setPPHFlatCost() dan setTaxPercentage() digunakan oleh reader untuk mengisi nilai pajak dari file konfigurasi. Pada onLand(), pemain diminta memilih antara membayar pajak flat atau persentase kekayaan. Kelas ini berelasi dengan Player untuk membaca saldo, total nilai properti, total nilai bangunan, serta mengurangi uang pemain. Jika pemain tidak mampu membayar, method mengembalikan hasil yang memicu alur kebangkrutan atau likuidasi oleh controller.

2.15. LuxuryGoodsTaxTile

JLuxuryGoodsTaxTile merupakan turunan dari TaxTile yang merepresentasikan Pajak Barang Mewah atau PBM. Kelas ini menyimpan flatCost_ sebagai nilai pajak tetap yang dibaca dari konfigurasi. Method onLand() langsung memotong saldo pemain sebesar nilai PBM. Jika saldo tidak cukup, kelas ini menghitung total aset pemain untuk menentukan apakah pemain perlu masuk ke alur likuidasi atau kebangkrutan. Dengan desain ini, tile hanya menentukan akibat awal dari pendaratan, sedangkan penyelesaian kebangkrutan tetap ditangani oleh controller.

2.16. FreeParkingTile

FreeParkingTile merupakan turunan dari SpecialTile yang merepresentasikan petak Bebas Parkir. Tanggung jawab kelas ini sederhana, yaitu menampilkan pesan dan mengembalikan OnLandResult::Done ketika pemain mendarat di petak tersebut. Tidak ada perubahan saldo, kepemilikan, atau status pemain. Kelas ini tetap dibuat sebagai kelas terpisah agar setiap jenis petak spesial memiliki representasi objek yang konsisten.

2.17. GoTile

GoTile merupakan turunan dari SpecialTile yang merepresentasikan petak mulai. Atribut salary_ menyimpan jumlah uang yang diterima pemain ketika mendarat di GO. Method getSalary() digunakan untuk mengambil nilai gaji, sedangkan onLand() menambahkan uang ke pemain melalui Player::addMoney(). Kelas ini berelasi langsung dengan Player karena efek utamanya adalah perubahan saldo pemain.

2.18. GoToJailTile

GoToJailTile merupakan turunan dari SpecialTile yang merepresentasikan petak Pergi ke Penjara. Method onLand() menampilkan pesan bahwa pemain harus dipindahkan ke penjara dan mengembalikan OnLandResult::TriggerMoveToJail. Pemindahan posisi dan perubahan status pemain tidak dilakukan langsung oleh tile, melainkan oleh controller. Dengan cara ini, tile hanya bertindak sebagai pemicu aturan, sedangkan perubahan state utama tetap dikendalikan oleh game logic.

2.19. PrisonTile

PrisonTile merupakan turunan dari SpecialTile yang merepresentasikan petak Penjara. Atribut fineCost_ menyimpan biaya denda untuk keluar dari penjara. Method getFineCost() digunakan oleh game logic ketika pemain ingin membayar denda. Pada implementasi onLand(), petak ini mengembalikan hasil yang memicu pemindahan atau status penjara. Relasi dengan Player dan controller diperlukan karena status jailed, posisi pemain, dan mekanisme keluar penjara dikelola pada level game logic.

2.20. SpecialTile

SpecialTile merupakan turunan dari ActionTile yang menjadi parent bagi petak spesial seperti GO, Penjara, Bebas Parkir, dan Pergi ke Penjara. Kelas ini digunakan untuk mengelompokkan petak-petak sudut atau petak khusus yang tidak termasuk properti, kartu, pajak, maupun festival. Setiap turunan SpecialTile memiliki perilaku onLand() yang berbeda sesuai aturan permainan.

2.21. ChanceTile

ChanceTile merupakan turunan dari CardTile yang merepresentasikan petak Kesempatan. Tanggung jawab utamanya adalah mendeteksi ketika pemain mendarat di petak Kesempatan dan mengembalikan OnLandResult::TakeChanceCard. Nilai tersebut kemudian diproses oleh controller untuk mengambil kartu dari deck Chance dan menjalankan efeknya. Dengan demikian, ChanceTile tidak mengatur isi kartu secara langsung, tetapi hanya menjadi pemicu alur pengambilan kartu.

2.22. CommunityChestTile

CommunityChestTile merupakan turunan dari CardTile yang merepresentasikan petak Dana Umum. Method onLand() menampilkan pesan bahwa pemain mendarat di petak Dana Umum dan mengembalikan OnLandResult::TakeCommunityChest. Relasi dengan controller penting karena eksekusi efek kartu tidak dilakukan langsung di tile, melainkan diteruskan ke sistem deck atau game logic yang bertanggung jawab terhadap kartu Dana Umum.

2.23. Deck

Deck<T> merupakan kelas generic yang digunakan untuk mengelola kumpulan kartu dalam permainan, baik kartu skill, chance, maupun community chest. Kelas ini menyimpan dua struktur utama yaitu drawPile_ (stack) dan discardPile_ (vector), serta capacity_ sebagai batas jumlah kartu. Method seperti initDeck() digunakan untuk inisialisasi deck, shuffleDeck() untuk mengacak kartu, drawDeck() untuk mengambil kartu dari deck, dan pushToDiscard() untuk membuang kartu yang telah digunakan. Kelas ini berelasi dengan berbagai tipe kartu melalui template <T>, sehingga mendukung reuse dan fleksibilitas sesuai prinsip generic programming.

2.24. CommunityChestCard

CommunityChestCard merupakan kelas yang merepresentasikan kartu Dana Umum. Kelas ini memiliki atribut instruction_ dan description_ yang menentukan efek kartu. Method useCommunityChest() menjalankan efek seperti menerima uang dari pemain lain atau membayar biaya tertentu. Kelas ini berelasi dengan banyak objek Player karena efeknya dapat melibatkan seluruh pemain dalam permainan. Selain itu, kelas ini digunakan oleh CommunityChestTile sebagai bagian dari mekanisme pengambilan kartu.

2.25. ChanceCard

ChanceCard merupakan kelas yang merepresentasikan kartu Kesempatan dalam permainan. Kelas ini memiliki atribut instruction_ yang menentukan jenis efek kartu dan description_ sebagai deskripsi teks. Method useChance() menjalankan efek kartu seperti berpindah ke stasiun terdekat, mundur beberapa petak, atau masuk penjara. Kelas ini berelasi dengan Player untuk mengubah posisi dan status pemain. Selain itu, kelas ini digunakan oleh ChanceTile sebagai sumber efek saat pemain mendarat di petak Kesempatan.

2.26. SkillCard

SkillCard merupakan kelas abstrak yang merepresentasikan kartu kemampuan spesial yang dapat dimiliki pemain. Kelas ini menyimpan atribut dasar seperti `name_` dan `description_` yang menjelaskan efek kartu. Method utama yang harus diimplementasikan oleh setiap turunan adalah `activate(Player& player)`, yang bertanggung jawab menjalankan efek kartu terhadap pemain. Kelas ini berelasi erat dengan kelas `Player` karena seluruh efek kartu akan memodifikasi state pemain seperti posisi, status, atau kondisi ekonomi. Dengan adanya inheritance, setiap jenis kartu kemampuan dapat memiliki perilaku berbeda namun tetap mengikuti kontrak yang sama melalui polymorphism.

2.27. DemolitionCard

DemolitionCard merupakan turunan dari SkillCard yang berfungsi untuk menghancurkan properti milik pemain lain. Method `activate()` akan mengumpulkan daftar properti lawan, meminta input pemain untuk memilih target, lalu mereset level bangunan dan efek festival dari properti tersebut. Kelas ini berelasi dengan `Player` dan `PropertyTile`, karena memodifikasi state properti milik pemain lain. Dengan demikian, kartu ini berperan dalam mekanisme kompetitif dan interaksi antar pemain.

2.28. DiscountCard

DiscountCard merupakan turunan dari SkillCard yang memberikan efek diskon terhadap transaksi pemain. Kelas ini memiliki atribut `discountPercent_` dan `remainingTurns_` yang mengatur besar diskon dan durasi efek. Method `activate()` akan mengaktifkan diskon pada pemain melalui method di kelas `Player`, sedangkan `consumeTurn()` digunakan untuk mengurangi durasi efek setiap giliran. Kelas ini berelasi dengan `Player` dalam pengelolaan status diskon dan mempengaruhi interaksi ekonomi seperti pembayaran sewa atau pembelian.

2.29. LassoCard

LassoCard merupakan turunan dari SkillCard yang digunakan untuk menarik pemain lain ke posisi pemain saat ini. Method `activate()` akan mencari pemain terdekat di depan dan memindahkan posisinya. Kelas ini berelasi dengan banyak objek `Player` sekaligus melalui method `getAllPlayers()`. Interaksi ini menunjukkan bahwa kartu tidak hanya mempengaruhi satu pemain, tetapi juga mempengaruhi state global permainan.

2.30. MoveCard

MoveCard merupakan turunan dari SkillCard yang berfungsi untuk memindahkan posisi pemain maju sejumlah langkah tertentu. Atribut `steps_` menyimpan jumlah langkah yang akan ditempuh, yang dapat ditentukan secara acak saat kartu dibuat. Method `activate()` akan

memanggil fungsi pada Player untuk mengubah posisi pemain. Kelas ini berelasi langsung dengan Player karena efeknya mempengaruhi posisi pada board. Desain ini memungkinkan integrasi dengan mekanisme movement pada game tanpa mengubah logika di kelas lain.

2.31. ShieldCard

ShieldCard merupakan turunan dari SkillCard yang memberikan perlindungan kepada pemain dari efek merugikan seperti pembayaran sewa atau pajak. Atribut `remainingTurns_` menyimpan durasi perlindungan. Method `activate()` akan mengaktifkan shield pada pemain melalui fungsi `activateShield()` di kelas Player. Method `consumeTurn()` digunakan untuk mengurangi durasi efek. Kelas ini berperan dalam mekanisme defensif permainan dan berelasi langsung dengan sistem pembayaran dan penalti.

2.32. TeleportCard

TeleportCard merupakan turunan dari SkillCard yang memungkinkan pemain berpindah ke petak mana pun di papan. Method `activate()` meminta input pengguna berupa ID petak tujuan dan kemudian memanggil `setPosition()` pada Player. Kelas ini berelasi dengan Board secara tidak langsung melalui representasi posisi pemain. Dengan desain ini, kartu memberikan fleksibilitas strategi karena pemain dapat memilih lokasi tujuan secara bebas.

2.33. Board

Board merupakan kelas yang bertanggung jawab menyimpan susunan seluruh petak permainan. Kelas ini memiliki atribut `tiles_` yang berisi kumpulan objek `Tile`, `size_` sebagai ukuran papan, serta posisi khusus seperti `startPosition_` dan `jailPosition_`. Method `addTile()` digunakan untuk menambahkan petak ke papan, sedangkan `getCurrentTile()`, `moveToNextTile()`, `getTileByCode()`, dan `getTileIndexByCode()` digunakan untuk mengambil petak berdasarkan posisi atau kode petak. Kelas ini berelasi dengan `Tile` sebagai container utama seluruh petak, baik property tile maupun action tile. Board juga membantu game logic dalam menentukan posisi GO, posisi penjara, pencarian petak berdasarkan kode, serta validasi kepemilikan seluruh color group melalui `isCompletedColourGroup()`. Dengan demikian, Board menjadi pusat representasi struktur papan permainan, tetapi tidak mengambil alih logika detail dari masing-masing petak karena perilaku pendaratan tetap berada di kelas turunan `Tile`.

2.34. Dice

Dice merupakan kelas yang merepresentasikan dua dadu dalam permainan. Kelas ini menyimpan nilai masing-masing dadu melalui `die1_` dan `die2_`. Method `roll()` digunakan untuk menghasilkan nilai acak antara 1 sampai 6 untuk kedua dadu, sedangkan `rollSettingan()`

digunakan untuk mengatur nilai dadu secara manual sesuai command ATUR_DADU. Method `getRollResult()` mengembalikan total nilai kedua dadu, sementara `getDie1()` dan `getDie2()` digunakan untuk menampilkan nilai masing-masing dadu. Method `isDouble()` digunakan untuk mengecek apakah kedua dadu memiliki nilai yang sama, yang berhubungan dengan mekanisme giliran tambahan dan aturan masuk penjara jika pemain mendapat double tiga kali. Kelas ini berelasi dengan game controller atau alur giliran pemain karena hasil dadu menentukan pergerakan Player pada Board.

2.35. Inventory

Inventory merupakan kelas yang bertanggung jawab menyimpan dan mengelola seluruh aset yang dimiliki oleh pemain, yaitu properti dan kartu kemampuan. Kelas ini menyimpan daftar `PropertyTile*` dan `SkillCard` yang dimiliki pemain, serta menghitung jumlah railroad dan utility yang dimiliki. Method seperti `addProperty()`, `removeProperty()`, `hasProperty()`, dan `clearProperties()` digunakan untuk mengatur kepemilikan properti pemain. Selain itu, method `addSkillCards()`, `removeSkillCard()`, `getSkillCardAt()`, dan `removeSkillCardAt()` digunakan untuk mengelola kartu skill di tangan pemain, termasuk validasi slot kartu maksimal tiga buah melalui exception `CardSlotFull`. Kelas ini berelasi langsung dengan Player, karena setiap pemain memiliki satu inventory sebagai tempat penyimpanan asetnya. Inventory juga berelasi dengan `PropertyTile`, `StreetTile`, `RailRoadTile`, `UtilityTile`, dan `SkillCard` untuk mendukung perhitungan kekayaan, jumlah properti, jumlah kartu, serta validasi color group dalam proses pembangunan.

2.36. Player

Player merupakan kelas yang merepresentasikan pemain dalam permainan. Kelas ini menyimpan identitas pemain seperti `id_` dan `username_`, kondisi ekonomi melalui `balance_`, posisi pada papan melalui `position_`, status pemain melalui `status_`, serta informasi tambahan seperti jumlah double roll, jumlah giliran di penjara, efek shield, dan efek discount. Kelas ini juga memiliki atribut `inventory_` yang menghubungkan pemain dengan properti dan kartu yang dimilikinya. Method seperti `addMoney()`, `deductMoney()`, `getBalance()`, `setStatus()`, `setPosition()`, dan `move()` digunakan untuk mengatur kondisi utama pemain selama permainan berjalan. Method jail seperti `isInJail()`, `incrementJailTurn()`, `resetJailTurn()`, dan `leaveJail()` mendukung mekanisme penjara, sedangkan method seperti `activateShield()`, `activateDiscount()`, dan `consumeDiscount()` mendukung efek kartu skill. Kelas ini berelasi erat dengan Inventory, `PropertyTile`, dan `SkillCard`, karena pemain dapat memiliki properti, menggunakan kartu, membayar uang, menerima uang, dan berpindah posisi di papan.

2.37. ConfigReader

`ConfigReader` atau `Reader` merupakan kelas pada data access layer yang bertanggung jawab membaca konfigurasi permainan dari file eksternal. Kelas ini membaca berbagai file konfigurasi seperti `property`, `action tile`, `railroad`, `utility`, `tax`, `special`, dan `misc`. Method `readSpecial()`

membaca nilai gaji GO dan denda penjara, `readTax()` membaca nilai PPH dan PBM, sedangkan `readAction()` membuat objek action tile seperti `GoTile`, `CommunityChestTile`, `ChanceTile`, `IncomeTaxTile`, `LuxuryGoodsTaxTile`, `FestivalTile`, `PrisonTile`, `GoToJailTile`, dan `FreeParkingTile`. Method `loadBoard()` menggabungkan property tile dan action tile, mengurutkannya berdasarkan ID, memvalidasi urutan petak, lalu membentuk objek Board. Kelas ini berelasi dengan banyak kelas tile karena bertugas melakukan instansiasi objek berdasarkan isi konfigurasi. Kelas ini juga berelasi dengan Board karena hasil akhir proses pembacaan konfigurasi adalah papan permainan yang siap digunakan oleh game logic

2.38. Monopoly

Monopoly adalah entry point utama permainan yang mengatur lifecycle game dari awal hingga akhir. Class ini menangani proses load config, registrasi pemain, inialisasi deck kartu, dice, serta memulai GameController untuk menjalankan game loop. Method seperti `startGame()`, `registerPlayers()`, `shufflePlayersTurn()`, dan `loadState()` mengatur setup awal dan restore game. Class ini menjadi orchestrator utama yang menghubungkan Board, Player, Deck, Reader, dan Controller.

2.39. Formater

ChanceCard merupakan kelas yang merepresentasikan kartu Kesempatan dalam permainan. Kelas ini memiliki atribut `instruction_` yang menentukan jenis efek kartu dan `description_` sebagai deskripsi teks. Method `useChance()` menjalankan efek kartu seperti berpindah ke stasiun terdekat, mundur beberapa petak, atau masuk penjara. Kelas ini berelasi dengan Player untuk mengubah posisi dan status pemain. Selain itu, kelas ini digunakan oleh ChanceTile sebagai sumber efek saat pemain mendarat di petak Kesempatan.

2.40. TransactionLogger

ChanceCard merupakan kelas yang merepresentasikan kartu Kesempatan dalam permainan. Kelas ini memiliki atribut `instruction_` yang menentukan jenis efek kartu dan `description_` sebagai deskripsi teks. Method `useChance()` menjalankan efek kartu seperti berpindah ke stasiun terdekat, mundur beberapa petak, atau masuk penjara. Kelas ini berelasi dengan Player untuk mengubah posisi dan status pemain. Selain itu, kelas ini digunakan oleh ChanceTile sebagai sumber efek saat pemain mendarat di petak Kesempatan.

2.41. Auction

Class Auction adalah engine utama untuk sistem lelang yang mengelola peserta, giliran bidding, validasi bid, kondisi pass, hingga menentukan pemenang dan mentransfer kepemilikan properti; class ini juga mendukung skenario khusus seperti lelang akibat kebangkrutan ke bank dan menggunakan pendekatan stateful untuk mengatur jalannya lelang.

2.42. AuctionCoordinator

Class AuctionCoordinator berfungsi sebagai perantara untuk menjalankan sistem lelang dengan memanggil class Auction, baik untuk kasus lelang biasa maupun akibat kebangkrutan pemain; class ini membantu memisahkan logika pemanggilan dari implementasi lelang itu sendiri.

2.43. GameConfig

GameConfig adalah class static yang menyimpan konfigurasi global permainan seperti:

- max turn
- starting balance
- salary GO
- jail fine

Class ini digunakan sebagai global state configuration yang diakses oleh berbagai class lain tanpa perlu instance.

2.44. GameController

GameController adalah pusat logika permainan (game engine). Class ini mengatur alur permainan seperti giliran pemain, pergerakan berdasarkan dadu, interaksi tile (onLand), serta command handling selama turn. Method penting meliputi playGame(), processTurn(), processMovement(), processNormalTurn(), dan processJailTurn().

Class ini juga mengintegrasikan berbagai coordinator seperti:

- AuctionCoordinator
- PropertyCoordinator
- SkillCardCoordinator

Sehingga GameController bertindak sebagai central brain yang menghubungkan semua sistem.

2.45. Liquidation

Class Liquidation merupakan komponen kompleks yang menangani proses likuidasi aset pemain saat tidak mampu membayar kewajiban, termasuk menghitung nilai maksimum aset, memberikan rekomendasi kombinasi aksi terbaik, memvalidasi constraint seperti aturan color group, serta mengeksekusi aksi seperti menjual properti, menggadaikan, atau menjual bangunan; class ini juga mengandung logika optimasi dalam memilih kombinasi aksi yang paling efisien.

2.46. PropertyCoordinator

Class PropertyCoordinator bertanggung jawab mengelola seluruh interaksi terkait properti, seperti pembayaran sewa, proses gadai dan tebus, pembangunan bangunan, serta efek tile seperti festival, chance card, dan community chest; selain itu class ini juga menangani kondisi khusus seperti kebangkrutan pemain terhadap pemain lain dengan bantuan LiquidationManager.

2.47. SkillCardCoordinator

Class SkillCardCoordinator mengatur seluruh mekanisme kartu kemampuan mulai dari mengambil kartu dari deck, menambahkan ke inventory pemain, membuang kartu ketika penuh, hingga penggunaan kartu dengan berbagai constraint seperti hanya boleh digunakan sekali per giliran atau tidak bisa digunakan saat kondisi tertentu seperti di penjara.

2.48. CommandInterface

ChanceCard merupakan kelas yang merepresentasikan kartu Kesempatan dalam permainan. Kelas ini memiliki atribut instruction_ yang menentukan jenis efek kartu dan description_ sebagai deskripsi teks. Method useChance() menjalankan efek kartu seperti berpindah ke stasiun terdekat, mundur beberapa petak, atau masuk penjara. Kelas ini berelasi dengan Player untuk mengubah posisi dan status pemain. Selain itu, kelas ini digunakan oleh ChanceTile sebagai sumber efek saat pemain mendarat di petak Kesempatan.

2.49. GameViewInterface

ChanceCard merupakan kelas yang merepresentasikan kartu Kesempatan dalam permainan. Kelas ini memiliki atribut instruction_ yang menentukan jenis efek kartu dan description_ sebagai deskripsi teks. Method useChance() menjalankan efek kartu seperti berpindah ke stasiun terdekat, mundur beberapa petak, atau masuk penjara. Kelas ini berelasi dengan Player untuk mengubah posisi dan status pemain. Selain itu, kelas ini digunakan oleh ChanceTile sebagai sumber efek saat pemain mendarat di petak Kesempatan.

2.50. Certificate

Class Certificate menyimpan informasi ekonomi dari suatu properti seperti harga beli, biaya pembangunan, nilai gadai, dan harga sewa, yang digunakan oleh PropertyTile dalam berbagai perhitungan ekonomi.

2.51. Exception

Class Exception sendiri menyimpan informasi dari keseluruhan hal yang menangani kesalahan saat program berjalan. Tujuannya agar program tidak langsung crash dan dapat mengatur control flow.

2.51.1. InvariantViolationException

InvariantViolationException merupakan turunan dari Exception yang digunakan untuk menangani pelanggaran terhadap invariant sistem, yaitu kondisi yang harus selalu benar selama permainan berlangsung.

2.51.1.1. InsufficientFundsException

Kelas ini ditugaskan untuk menangani ketika player tidak memiliki cukup uang.

a. RentPaymentFailed

Kelas yang menangani kegagalan pembayaran sewa kepada pemain lain.

b. PropertyPaymentFailed

Kelas yang menangani kegagalan dalam pembelian properti.

c. CardPaymentFailed

Kelas yang menangani kegagalan pembayaran akibat efek kartu.

d. PBMPaymentFailed

Kelas yang menangani kegagalan pembayaran Pajak Barang Mewah (PBM).

e. PPHPaymentFailed

Kelas yang menangani kegagalan pembayaran Pajak Penghasilan (PPH).

2.51.1.2. CardSlotFull

Kelas yang menangani kondisi ketika jumlah kartu yang dimiliki pemain melebihi batas maksimum.

2.51.2. ConfigException

ConfigException merupakan turunan dari Exception yang digunakan untuk menangani kesalahan yang berkaitan dengan konfigurasi sistem.

2.51.2.1. LoadConfigException

Kelas yang menangani kegagalan saat memuat file konfigurasi.

2.51.2.2. NewConfigException

Kelas yang menangani kegagalan saat membuat konfigurasi baru.

2.51.2.3. SaveConfigException

Kelas yang menangani kegagalan saat menyimpan data permainan.

2.51.3. SessionException

SessionException merupakan turunan dari Exception yang digunakan untuk menangani kesalahan yang berkaitan dengan jalannya sesi permainan (game session).

2.51.3.1. BankruptcyException

Kelas yang menangani kondisi ketika pemain dinyatakan bangkrut.

2.51.3.2. InvalidCommandAction

Kelas yang menangani kesalahan pada input atau eksekusi perintah.

3. Justifikasi Desain

Desain yang dibuat berusaha mengimplementasikan pattern MVC yang terdiri dari Model, Controller, dan juga View. Hal ini bertujuan agar memudahkan proses pemisahan tanggung jawab antarkelas yang ada. Dalam pattern ini, model bertanggungjawab mengelola data dan juga state permainan, misalnya kelas Player, Board, Tile, Dice, Card, dan lain-lain. Di lain sisi, Controller merupakan pengatur alur permainan yang menjadi penghubung juga antara view dan juga model, misalnya saja GameController yang dibantu juga dengan kelas-kelas seperti Liquidation, PropertyCoordinator, AuctionCoordinator, dan lain-lain. View merupakan kelas yang bertanggung jawab untuk menampilkan pesan kepada pengguna dan berperan untuk menerima input dari pengguna. Diagram kelas yang kami buat saat ini berubah dibandingkan diagram kelas yang telah kami buat di milestone 1. Hal ini dikarenakan kebutuhan yang meningkat untuk memecah suatu kelas ataupun menambahkan kelas untuk membantu berjalannya pengembangan program. Kelebihan dari diagram kelas yang telah kami buat saat ini ialah lebih memisahkan tanggung jawab antara kelas dan juga lebih mengutamakan dependency injection untuk mengurangi coupling antar kelas. Di lain sisi, kekurangan diagram kelas yang dipilih ialah masih ada beberapa kelas yang memiliki tanggung jawab cukup banyak meskipun sudah berusaha dibagikan tanggung jawabnya ke kelas lainnya juga.

Kendala yang kami hadapi ialah sulitnya membuat keputusan desain terutama ketika harus memikirkan tradeoff yang terjadi. Selain itu, susahnya mencari referensi yang konsisten juga cukup berpengaruh karena ketika banyak referensi yang berbeda-beda dapat membuat bingung terkait mana yang dapat dijadikan acuan.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Konsep Inheritance & Polymorphism digunakan untuk membangun hubungan antar kelas serta memberikan fleksibilitas dalam penanganan objek. Konsep ini bisa dilihat pada kelas parent dan child dari kelas Tile. Untuk source code sendiri, terdapat beberapa hubungan Inheritance dan Polymorphism. Berikut merupakan beberapa hubungan beserta penjelasan yang lebih lengkap.

1. Kelas Tile
 - a. Inheritance


```

class Tile { };
class PropertyTile : public Tile { };
class ActionTile : public Tile { };
class TaxTile : public Tile { };
class ChanceTile : public Tile { };

```

b. Polymorphism

Setiap tile memiliki aksi yang berbeda

```

class Tile {
public:
    virtual void onLand(Player& p) = 0;
};

```

Berikut merupakan beberapa gambaran terkait polymorphism.

```

class PropertyTile : public Tile {
public:
    void onLand(Player& p) override {
        // beli / bayar sewa
    }
};

class TaxTile : public Tile {
public:
    void onLand(Player& p) override {
        // bayar pajak
    }
}

```

```
};
```

2. Exception

a. Inheritance

```
class Exception { };
class InvariantViolationException : public Exception { };
class ConfigException : public Exception { };
class SessionException : public Exception { };
```

b. Polymorphism

```
try {
    // operasi game
} catch (Exception& e) {
    cout << e.what();
}
```

Hal ini juga diimplementasikan pada kelas Exception di General Exception yang memiliki turunan kelas yang banyak, berikut merupakan cuplikan kodenya.

```
#pragma once
#include <iostream>
#include "../injector/injector.hpp"
```

```
class Player;  
class CommandInterface;  
class GameViewInterface;  
  
enum class OnLandResult {  
    Done,  
    TriggerAuction,  
    TakeChanceCard,  
    TakeCommunityChest,  
    TriggerBankruptcyAuction, //  
    Festival,  
    TriggerMoveToJail,  
    TriggerBankruptcyFlow,  
    TriggerCanPayRentUsingCash,  
    TriggerTryToPayRent  
    // tambahin lagi
```

```
};
```

```
class Tile {
```

```
protected:
```

```
    int tileID_;
```

```
    std::string letterCode_; // kode petak dari properti
```

```
    std::string tileName_;
```

```
    std::string colourBlock_;
```

```
public:
```

```
    // Ctor & Dtor
```

```
    Tile(int tileID, std::string letterCode,
```

```
        std::string tileName, std::string colourBlock);
```

```
    virtual ~Tile();
```

```
    virtual OnLandResult onLand(Player& p, CommandInterface& command, GameViewInterface& view) = 0;
```

```
virtual void getDetail() const;

// Getter

int getTileID() const;

std::string getLetterCode() const;

std::string getTileName() const;

std::string getColourBlock() const;

};
```

4.2. Method/Operator Overloading

Penerapan operator overloading kami gunakan pada kelas player, yaitu pada operator > dan juga operator ==. Di mana operator ini digunakan untuk membandingkan mana player yang menempati posisi yang lebih layak menang (>) dibandingkan player lainnya dan juga melihat player mana yang setara untuk menjadi pemenang bersama (==). Berikut ini implementasi operator overloading yang dilakukan.

```
bool Player::operator>(const Player& other) const {
    if (getBalance() > other.getBalance()) {
        return true;
    }
}
```

```
if (getPropertyCount() > other.getPropertyCount()) {  
    return true;  
}  
  
if (getCardCount() > other.getCardCount()) {  
    return true;  
}  
  
return false;  
}  
  
bool Player::operator==(const Player& other) const {  
    return getBalance() == other.getBalance() &&  
        getPropertyCount() == other.getPropertyCount() &&  
        getCardCount() == other.getCardCount();  
}
```

Berikut ini contoh pemakaian operator overloading ketika akan mencari pemenang dari game Monopoly yang berjalan

```
.....  
  
    Player* topPlayer = candidates[0];  
    for (size_t i = 1; i < candidates.size(); i++) {  
        if (*candidates[i] > *topPlayer) {  
            topPlayer = candidates[i];  
        }  
    }  
  
    std::vector<Player*> winners;  
    for (auto* p : candidates) {  
        if (*p == *topPlayer) { // Memanggil operator == yang kita buat  
            winners.push_back(p);  
        }  
    }  
    .....  
}
```

4.3. Template & Generic Classes

Deck<T> merupakan kelas generic yang digunakan untuk mengelola kumpulan kartu dalam permainan, seperti kartu skill, chance, maupun community chest. Penggunaan generic memungkinkan setiap deck menyimpan tipe objek yang berbeda secara fleksibel dan type-safe tanpa perlu membuat kelas terpisah untuk setiap jenis kartu.

```
// Di file Deck.hpp

#pragma once

#include <iostream>
#include <vector>
#include <stack>
#include <algorithm>
#include <memory>
#include <random>
#include <utility>

template <typename T>
class Deck {
    private:
        std::stack<std::unique_ptr<T>> drawPile_;
        std::vector<std::unique_ptr<T>> discardPile_;
        int capacity_;

    public:
        Deck(int capacity = 0);

        // Inisialisasi deck dengan kumpulan kartu berpemilik
        void initDeck(std::vector<std::unique_ptr<T>> initialCards);

        // Mengambil kartu (Otomatis reshuffle jika drawPile kosong)
        std::unique_ptr<T> drawDeck();
};
```



```

// Memasukkan kartu kembali ke tumpukan buangan
void pushToDiscard(std::unique_ptr<T> item);

// Mengacak kartu menggunakan Mersenne Twister
void shuffleDeck();

int getDrawCount() const;
int getDiscardCount() const;
};

```

4.4. Exception

Kelas ini merupakan kelas yang mengatur control flow dari sistem ketika terjadi kesalahan atau kondisi tidak normal selama eksekusi program. Dengan adanya exception, program dapat menangani error secara terstruktur tanpa menghentikan seluruh proses secara tiba-tiba. Salah satu contoh dari penerapan exception ini ada di kelas PropertyCoordinator.

```

// Di PropertyCoordinator.hpp
void PropertyCoordinator::processTakeCommunityChest(Player& player) {
    CommunityChestCard card = CommunityChestCard::randomCard();
    view_.showMessage("[COMMUNITY CHEST] " + card.getDescription() + "\n");

    switch (card.getInstruction()) {
        case CommunityChestCard::BirthdayCollect100FromEachPlayer:
            for (auto& otherPtr : players_) {
                Player* other = otherPtr.get();

```

```
        if (other == nullptr || other == &player || other->isBankrupt()) {
            continue;
        }

        other->deductMoney(100);
        player.addMoney(100);
    }
    break;

case CommunityChestCard::DoctorFeePay700:
    try {
        player.deductMoney(700);
    }
    catch(const InsufficientFundsException& e) {
        throw CardPaymentFailedException(101, "Gagal memakai Kartu DoctorFeePay700 karena saldo tidak
mencukupi\n");
    }

    break;

case CommunityChestCard::ElectionPay200ToEachPlayer : {
    int sumMoney = 0;
    for (auto& otherPtr : players_) {
        Player* other = otherPtr.get();
        if (other == nullptr || other == &player || other->isBankrupt()) {
            continue;
        }
        sumMoney += 200;
    }
}
```

```

        if (player.getBalance() < sumMoney) {
            throw CardPaymentFailedException(101, "Gagal memakai kartu ElectionPay200ToEachPlayer\n");
        }

        // harusnya udh aman di sini
        for (auto& otherPtr : players_) {
            Player* other = otherPtr.get();
            if (other == nullptr || other == &player || other->isBankrupt()) {
                continue;
            }
            player.deductMoney(200);
            other->addMoney(200);
        }
    }
    break;

default:
    break;
}
}

```

5. C++ Standard Template Library

Pada tugas besar kali ini, tim memanfaatkan C++ Standard Template Library (STL) untuk meningkatkan efisiensi, keamanan, dan keterbacaan kode memanfaatkan RAIL. Struktur data seperti `std::vector` digunakan untuk menyimpan kumpulan objek dinamis seperti daftar pemain dan tile karena menyediakan akses indeks yang cepat serta fleksibilitas ukuran. `std::map` digunakan untuk mengelompokkan data seperti properti berdasarkan grup warna, sehingga mempermudah proses pencarian dan pengolahan data. Stack juga digunakan pada kelas deck

```
//Cuplikan dari Monopoly.h
std::unique_ptr<GameViewInterface> view_;
std::unique_ptr<CommandInterface> command_;
static inline int maxTurn_;
static inline int initialBalance_;

std::vector<std::unique_ptr<Player>> registerPlayers(size_t numOfPlayers);
void shufflePlayersTurn(std::vector<std::unique_ptr<Player>>& players_);
std::unique_ptr<Board> loadConfig();

// Atribut dari kelas Deck.hppp
std::stack<std::unique_ptr<T>> drawPile_;
std::vector<std::unique_ptr<T>> discardPile_;
int capacity_;
```

5.1. Konsep OOP lain

5.1.1. Abstract Base Class

Terdapat sejumlah penggunaan abstract base class dalam pembuatan game Nimonspoli, misalnya saja kelas Tile yang berperan sebagai Abstract Base Class bagi turunan-turunannya, kelas GameViewInterface yang juga menjadi Abstract Base Class bagi kelas View sehingga memungkinkan penggunaan View secara CLI maupun GUI (meskipun GUI belum sempat diimplementasikan), kelas CommandInterface yang juga menjadi Abstract Base Class bagi kelas Command (untuk meminta input) baik secara CLI maupun GUI. Penggunaan abstract base class membantu memastikan kelas-kelas turunannya mempunyai method-method bersangkutan yang menjadi kontrak, misalkan saja pada kelas Tile, terdapat method **OnLand**

yang tugasnya memproses ketika terdapat player yang berpindah ke tile bersangkutan sesuai dengan tipe tile nantinya, baik street tile, go tile, dan lain-lain.

```
class Tile {  
    protected:  
        int tileID_;  
        std::string letterCode_;  
        std::string tileName_;  
        std::string colourBlock_;  
  
    public:  
        Tile(int tileID, std::string letterCode,  
            std::string tileName, std::string colourBlock);  
        virtual ~Tile();  
        virtual OnLandResult onLand(Player& p, CommandInterface& command,
```

```
GameViewInterface& view) = 0;  
    virtual void getDetail() const;  
    int getTileID() const;  
    std::string getLetterCode() const;  
    std::string getTileName() const;  
    std::string getColourBlock() const;  
};
```

5.1.2. Aggregation (tambahin lagi)

Salah satu penerapan agregasi di dalam program ialah hubungan antara Board dengan Tile, di mana di dalam board terdapat

```
std::vector<std::unique_ptr<Tile>> tiles_;
```

yang menandakan bahwa board has-a Tile, akan tetapi ketika Board dihapus, Tile tetap dapat berdiri sendiri.

5.1.3. Composition

Implementasi dari Konsep Composition adalah hubungan Monopoly dengan Player dan juga Board, di mana hubungan Monopoly memiliki 2 hingga 4 Player dan juga sebuah Board di dalamnya. Dengan kata lain, setiap sesi permainan pasti memiliki setidaknya 2 pemain dan juga sebuah papan monopoli. Terdapat juga beberapa hubungan Composition pada kelas lain seperti hubungan antara Certificate dengan PropertyTile, Player dengan Inventory, AuctionCoordinator dengan Auction serta GameController dengan Player, AuctionCoordinator, PropertyCoordinator, dan SkillCardCoordinator. Hal ini bisa dilihat dari atribut dari masing-masing kelas, misal pada cuplikan kode berikut pada file Monopoly.h.

```
// Monopoly.h

#pragma once
#include "models/player/Player.hpp"
#include "models/board/Board.hpp"
#include "controllers/GameController.hpp"
#include "models/dice/Dice.hpp"
#include <iostream>
#include <memory>
#include "views/GameViewInterface.hpp"
#include <algorithm>
#include <random>
#include "models/card/skillcard/SkillCard.hpp"
```

```
class Monopoly {
private:
    std::unique_ptr<GameViewInterface> view_;
    std::unique_ptr<CommandInterface> command_;
    static inline int maxTurn_;
    static inline int initialBalance_;

    std::vector<std::unique_ptr<Player>> registerPlayers(size_t numOfPlayers);
    void shufflePlayersTurn(std::vector<std::unique_ptr<Player>>& players_);
    std::unique_ptr<Board> loadConfig();

public:
    bool loadState(std::unique_ptr<Board>& board,
std::vector<std::unique_ptr<Player>>& players, int& latestTurn);
    Monopoly(std::unique_ptr<GameViewInterface> view,
std::unique_ptr<CommandInterface> command);
    ~Monopoly() = default;

    static void setMaxTurn(int maxTurn);
    static void setInitialBalance(int initialBalance);
    static int getMaxTurn();

    void startGame();
};
```


6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
CETAK_PAPAN	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
LEMPAR_DADU	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
ATUR_DADU	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
CETAK_AKTA	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
CETAK_PROPERTI	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
BELI	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
BAYAR_SEWA	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110

BAYAR_PAJAK	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
GADAI	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
TEBUS	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
BANGUN	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
LELANG	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
FESTIVAL	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
BANGKRUT DAN AMBIL ALIH ASET	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
SIMPAN	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
MUAT	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
CETAK_LOG	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
MENANG	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
KESEMPATAN DAN DANA UMUM	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110

GUNAKAN KEMAMPUAN	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110
DROP KARTU KEMAMPUAN	13524008, 13524030, 13524074,13524096, 13524110	13524008, 13524030, 13524074,13524096, 13524110

Lampiran

Lampiran A. Form Asistensi

 IF2010_TB1_Asistensi_STI.docx.pdf